

Principle and Practice of Bidirectional Programming in BiGUL

Zhenjiang Hu, Hsiang-Shang Ko

National Institute of Informatics, Japan
{hu,hsiang-shang}@nii.ac.jp

Abstract. This material will be used for the Summer School on Bidirectional Transformation to be held in Oxford in July, 2016.

1 Installation

The latest version of BiGUL has been released to Hackage. To install the package, just run the following two commands.

```
> cabal update
> cabal install bigul
```

Now you can easily check whether BiGUL is correctly installed. First, create a simple file called “test.hs” with the following content for importing BiGUL modules.

```
import Generics.BiGUL.Error
import Generics.BiGUL.AST
import Generics.BiGUL.Interpreter
import Generics.BiGUL.TH
```

Then load it using the GHC interactive interpreter.

```
> ghci test.hs
GHCi, version 7.10.3: http://www.haskell.org/ghc/  :? for help
Prelude>~/Repositories/prl_tokyo/bigul/SummerSchool16> ghci test.hs
GHCi, version 7.10.3: http://www.haskell.org/ghc/  :? for help
[1 of 1] Compiling Main                ( test.hs, interpreted )
Ok, modules loaded: Main.
```

If you see the above message, congratulations for your successful installation.

2 Basics

Intuitively, we can think of a bidirectional BiGUL program:

$$bx :: BiGUL\ s\ v$$

as describing how to manipulate a state consisting of a source component of type s and a view component of type v ; the goal is to embed all information in the view to proper places in the source. For each bx , we can run it forwardly by calling *get* and backwardly by calling *put*.

$$\begin{aligned} \textit{get } bx &:: s \rightarrow v \\ \textit{put } bx &:: s \rightarrow v \rightarrow s \end{aligned}$$

where *get bx* is a function mapping a source to a view, while *put bx* accepts an original source and an update view and returns an updated source.

The core of BiGUL consists of a small number of combinators for constructing bidirectional transformation through development of well-behaved putback transformations.

2.1 Skip

The first one is the simple primitive

$$\textit{Skip} :: \textit{BiGUL } s ()$$

Its putback semantics is to skip any change on the source for the unit view. One can test this using `ghci` after loading the `test.hs` by

```
*Main> put Skip 5 ()
Right 5
```

which means that *Skip* putbacks the unit view `()` on the source 5 and gets the same source 5. In fact, each well-behaved putback transformation is equipped with a unique *get* for doing forward transformation.

```
*Main> get Skip 5
Right ()
```

It reads that doing forward transformation for *Skip* on the source 5 gives the unit view `()`.

2.2 Replace

The second one is

$$\textit{Replace} :: \textit{BiGUL } s s$$

which is to use the view to replace the source:

```
*Main> put Replace 1 100
Right 100
```

which uses the view 100 to replace the source 1 and gets a new source 100.

2.3 Product: Prod

If we want to putback a view pair (v_1, v_2) to a source pair (s_1, s_2) , we may write $bx1 \text{ Prod } bx2$, a product of two putback transformation, to putback v_1 to s_1 with $bx1$ and v_2 to s_2 with $bx2$.

$$Prod :: BiGUL\ s_1\ v_1 \rightarrow BiGUL\ s_2\ v_2 \rightarrow BiGUL\ (s_1, s_2)\ (v_1, v_2)$$

For instance, we can combine *Skip* and *Replace* to putback a view pair to a source pair.

```
*Main> put (Skip 'Prod' Replace) (5,1) ((),100)
Right (5,100)
```

We may deal with more complicated structure using nested *Prod*:

```
*Main> put ((Skip 'Prod' Replace) 'Prod' Replace) ((5,1),2) ((((),100),200)
Right ((5,100),200)
```

2.4 Source/View Rearrangement

So far, the source and the view must be of the same structure. What if we wish to putback a pair view (v_1, v_2) to a complicated source, say $((s0, s_1), s_2)$, where we hope to replace s_1 by v_1 and s_2 by v_2 ? BiGUL provide a way of preprocessing of rearranging either the source or the view through a natural transformation τ .

$$\begin{aligned} \$ (rearrS \ [\ \tau :: s_1 \rightarrow s_2 \]) &:: BiGUL\ s_2\ v \rightarrow BiGUL\ s_1\ v \\ \$ (rearrV \ [\ \tau :: v_1 \rightarrow v_2 \]) &:: BiGUL\ s\ v_2 \rightarrow BiGUL\ s\ v_1 \end{aligned}$$

Returning to the above problem, we can solve it by defining the following putback transformation

$$\begin{aligned} putPairOverNPair &:: BiGUL\ ((s0, s_1), s_2)\ (s_1, s_2) \\ putPairOverNPair &= \$ (rearrS \ [\ \lambda((s0, s_1), s_2) \rightarrow (s_1, s_2) \]) \ Replace \end{aligned}$$

and have

```
*PBasic> put putPairOverNPair ((5,1),2) (100,200)
Right ((5,100),200)
```

In fact, it is possible to define *putPairOverNPair* by rearranging the view too.

$$\begin{aligned} putPairOverNPair' &:: BiGUL\ ((s0, s_1), s_2)\ (s_1, s_2) \\ putPairOverNPair' &= \$ (rearrV \ [\ \lambda(v_1, v_2) \rightarrow ((((), v_1), v_2) \]) \ \$ \\ &\quad (Skip\ 'Prod'\ Replace)\ 'Prod'\ Replace \end{aligned}$$

The mechanism of source/view rearrangement makes it possible to process algebraic data structures such as lists and trees. The following example describes using view v to replace the first element of a nonempty source.

$$\begin{aligned} pHead &:: BiGUL\ [s]\ s \\ pHead &= \$ (rearrS \ [\ \lambda(s: _) \rightarrow s \]) \ Replace \end{aligned}$$

```
*PBasic> put pHead [1,2,3,4] 100
Right [100,2,3,4]
```

We may go further to define a general putback transformation to replace the i th element of a source list with a view v .

$$\begin{aligned}
pNth &:: \mathbb{Z} \rightarrow BiGUL [s] s \\
pNth \ i &= \text{if } i \equiv 0 \text{ then } pHead \\
&\quad \text{else } \$ (rearrS [\lambda(x : xs) \rightarrow (x, xs)]) \$ \\
&\quad \quad \$ (rearrV [\lambda v \rightarrow ((), v)]) \$ \\
&\quad \quad Skip 'Prod' pNth (i - 1)
\end{aligned}$$

```
*PBasic> put (pNth 3) [1..10] 100
Right [1,2,3,100,5,6,7,8,9,10]
```

Note that any putback function is equipped with a *get* function (which may be partial). So for $pNth$, the corresponding *get* function is actually the familiar *take* function.

```
*PBasic> get (pNth 3) [1..10]
Right 4
```

2.5 Case

The *Case* combinator is for case analysis. The basic structure is as follows.

$$\begin{aligned}
Case & [\$ (normal [\text{cond1} :: s \rightarrow v \rightarrow Bool]) (bx1 :: BiGUL s v), \\
&\quad \$ (normal [\text{cond2} :: s \rightarrow v \rightarrow Bool]) (bx2 :: BiGUL s v), \\
&\quad \dots, \\
&\quad \$ (normal [\text{condn} :: s \rightarrow v \rightarrow Bool]) (bxn :: BiGUL s v), \\
&\quad \$ (adaptive [\text{cond1}' :: s \rightarrow v \rightarrow Bool]) \$ (f1 :: s \rightarrow v \rightarrow s), \\
&\quad \$ (adaptive [\text{cond2}' :: s \rightarrow v \rightarrow Bool]) \$ (f2 :: s \rightarrow v \rightarrow s), \\
&\quad \dots, \\
&\quad \$ (adaptive [\text{condm}' :: s \rightarrow v \rightarrow Bool]) \$ (fm :: s \rightarrow v \rightarrow s) \\
&\quad] \\
&:: BiGUL s v
\end{aligned}$$

It contains a sequence of cases. For each case, it is either *normal* or *adaptive*. For the normal case, if the condition is satisfied, a corresponding putback transformation is applied. For the adaptive case, if the condition is satisfied, a function is used to update the source with the view so that for the next step one of the normal cases can be applied. Note that if adaptation does not lead the source and the view to a normal case, an error will be reported at runtime.

As a simple example, consider using the view to update all the elements in the source list. To do so, we can use *Case*.

$$\begin{aligned}
replaceAll &:: Eq s \Rightarrow BiGUL [s] s \\
replaceAll &= Case [
\end{aligned}$$

```

$ (normal [|  $\lambda s\ v \rightarrow \text{length } s \equiv 1$  |]) $
$ (rearrS [|  $\lambda [x] \rightarrow x$  |]) Replace,
$ (normal [|  $\lambda s\ v \rightarrow \text{length } s > 1$  |]) $
$ (rearrS [|  $\lambda (x : xs) \rightarrow (x, xs)$  |]) $
$ (rearrV [|  $\lambda v \rightarrow (v, v)$  |]) $
  Replace 'Prod' replaceAll,
$ (adaptive [|  $\lambda s\ v \rightarrow \text{length } s \equiv 0$  |]) $
   $\lambda s\ v \rightarrow [\perp]$ 
]

```

```

*PBasic> put replaceAll [] 100
Right [100]
*PBasic> put replaceAll [1..10] 100
Right [100,100,100,100,100,100,100,100,100,100]

```

2.6 Dependency: Dep

Sometimes, a view may contain derived value that is computed from other part of the view and not allowed to be changed. For instance, for the view $(x, x + 1)$, the second component can be computed from the first by increasing it by 1. To capture this, BiGUL provides

$$\text{Dep} :: \text{Eq } v' \Rightarrow \text{BiGUL } a\ v \rightarrow (a \rightarrow v \rightarrow v') \rightarrow \text{BiGUL } a\ (v, v')$$

to describe this intention. We may, for example, define

```

replaceAll2 :: BiGUL [Z] (Z, Z)
replaceAll2 = Dep replaceAll ( $\lambda s\ v \rightarrow v + 1$ )

```

to replace all elements of the source by the first component of the view, while the second component of the view is a derived one that should be one bigger than the first one.

```

*PBasic> put replaceAll2 [1..10] (100,101)
Right [100,100,100,100,100,100,100,100,100,100]
*PBasic> put replaceAll2 [1..10] (100,200)
Left dependency mismatch

```

As seen in the last running of *put*, it reports an error because in the view (100, 200), 200 is not one bigger than 100.

2.7 Composition

Composition of two putback transformations follows that of two bidirectional transformations. This is because each putback transformation is essentially a bidirectional transformation.

$$\text{Compose} :: \text{BiGUL } a \ u \rightarrow \text{BiGUL } u \ b \rightarrow \text{BiGUL } a \ b$$

As an example, consider

$$\begin{aligned} \text{pHead2} &:: \text{BiGUL } [[a]] \ a \\ \text{pHead2} &= \text{pHead} \text{ 'Compose' } \text{pHead} \end{aligned}$$

```
*PBasic> put pHead2 [[1,2],[3,4,5],[]] 100
Right [[100,2],[3,4,5],[]]
```

2.8 Utilities

BiGUL provides some useful functions for building putback transformations. One interesting one is *emb* that can safely embed a pair of well-behaved *get* and *put* a putback transformation into our context. *emb* itself is defined as follows.

$$\begin{aligned} \text{emb} &:: \text{Eq } v \Rightarrow (s \rightarrow v) \rightarrow (s \rightarrow v \rightarrow s) \rightarrow \text{BiGUL } s \ v \\ \text{emb } g \ p &= \text{Case } [\$ (\text{normal } [\lambda x \ y \rightarrow g \ x \equiv y \]) \$ \\ &\quad \$ (\text{rearrV } [\lambda x \rightarrow ((), x) \]) \$ \\ &\quad \text{Dep Skip } (\lambda x \ () \rightarrow g \ x) \\ &\quad , \$ (\text{adaptive } [\backslash _ \rightarrow \text{True} \]) \\ &\quad \quad p \\ &\quad] \end{aligned}$$

As an application of *emb*, we may define a useful putback functions for reflecting the sum to a pair.

$$\begin{aligned} \text{distSum} &:: \text{BiGUL } (\mathbb{Z}, \mathbb{Z}) \ \mathbb{Z} \\ \text{distSum} &= \text{emb } g \ p \\ \text{where } g \ (x, y) &= x + y \\ p \ (x, y) \ v &= (v - y, y) \end{aligned}$$

```
*PBasic> put distSum (1,2) 100
Right (98,2)
*pBasic> get distSum (1,2)
Right 3
```

3 Bidirectional Programming on Lists

In this section, we show many list functions can be bidirectionalized using BiGUL. To show the correspondence with the original we prefix the original forward function names with "lens". Note that the forward function can be automatically derived from the new putback transformation by calling *get*.

3.1 lensFoldr

foldr is a simple but useful higher order function on lists.

$$\begin{aligned} foldr\ f\ e\ [] &= e \\ foldr\ f\ e\ (x:xs) &= f\ x\ (foldr\ f\ e\ xs) \end{aligned}$$

Many interesting functions can be defined in *foldr*

```

sum = foldr (+) 0
map f = foldr (\a r → f a : r) []
filter p = foldr (\a r → if p a then a : r else r) []
reverse p = foldr (\a r → r ++ [a]) []
sort = foldr insert []

```

First, we develop a putback transformation for *foldr*.

$$\begin{aligned}
& \text{lenFoldr} :: \text{BiGUL } (a, b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow \text{BiGUL } ([a], b) \rightarrow b \\
& \text{lenFoldr } bx \text{ pv} = \\
& \quad \text{Case } [\$ (\text{adaptive } [[\lambda(x, y) \rightarrow \text{pv } v \wedge \text{length } x \neq 0]]) \$ \\
& \quad \quad \lambda(x, y) \rightarrow ([, y) \\
& \quad , \$ (\text{normalS } [[\lambda(s, e) \rightarrow \text{length } s \equiv 0]]) \$ \\
& \quad \quad \$ (\text{rearrV } [[\lambda v \rightarrow ((, v)]]) \$ \\
& \quad \quad \quad \$ (\text{update } [p \mid ((, v)] [p \mid (-, v)] [d \mid v = \text{Replace }]]) \\
& \quad , \$ (\text{normalSV } [p \mid -] [p \mid -]) \$ \\
& \quad \quad \$ (\text{rearrS } [[\lambda((x : xs), e) \rightarrow (x, (xs, e))]]) \\
& \quad \quad (\text{Replace 'Prod' lenFoldr } bx \text{ pv}) \text{ 'Compose' } bx \\
&]
\end{aligned}$$

lengFoldr *bx pv* is to synchronize source (xs, v) with view v' . If v' satisfies pv , it will stop. Otherwise, it will recursively apply *bx* rightwards over the elements of *xs*.

$$\begin{aligned}
lensReverse &:: BiGUL [a] [a] \\
lensReverse &= \$ (rearrS [\lambda s \rightarrow (s, [])]) \$ \\
&\quad lensFoldr \ lensSnoc \ null \\
lensSnoc &:: BiGUL (a, [a]) [a] \\
lensSnoc &= Case [\$ (normal [\lambda s \ v \rightarrow length \ v \equiv 1]) \$ \\
&\quad \$ (rearrV [\lambda [v] \rightarrow (v, [])]) \ Replace, \\
&\quad \$ (normal [\lambda (-, s) \ v \rightarrow length \ s > 0]) \$ \\
&\quad \$ (rearrS [\lambda (x, y : ys) \rightarrow (y, (x, ys))]) \$ \\
&\quad \$ (rearrV [\lambda (v : vs) \rightarrow (v, vs)]) \$ \\
&\quad \quad \quad Replace \ 'Prod' \ lensSnoc, \\
&\quad \$ (adaptive [\lambda (-, s) \ v \rightarrow null \ s]) \$ \\
&\quad \quad \quad \lambda (x, s) \rightarrow (x, [\perp])]
\end{aligned}$$

```

*PList> put lensSnoc (1,[2,3,4]) [10,11,12,13]
Right (13,[10,11,12])
*PList> put lensSnoc (1,[2,3,4]) [10,11,12,13,14]
Right (14,[10,11,12,13])
*PList> put lensSnoc (1,[2,3,4]) [10,11]
Right (11,[10])
*PList> get lensSnoc (100,[1..10])
Right [1,2,3,4,5,6,7,8,9,10,100]

*PList> put lensReverse [1..10] [100..105]
Right [105,104,103,102,101,100]
*PList> put lensReverse [1..10] [100..115]
Left either value mismatch
*PList> get lensReverse [1..10]
*PList> get lensReverse [1..10]
Right [10,9,8,7,6,5,4,3,2,1]

```

3.2 Efficiency Issue: *lensFoldr*

A close look at the definition of *lensFoldr* reveals that it contains many redundant computations because of the use of *Compose* that calls *get* as many times as the number of calls of *Compose*. Put it more concretely,

$$\text{put } (\text{lensFoldr } bx \text{ (const True)}) ([x1, x2, \dots, xn], e) v$$

would call

$$\begin{aligned} &\text{get } (\text{lensFoldr } bx \text{ (const True)}) ([x2, \dots, xn], e), \\ &\dots, \\ &\text{get } (\text{lensFoldr } bx \text{ (const True)}) ([], e) \circ \end{aligned}$$

3.3 LensScanr

3.4 LensMap

$$\begin{aligned} \text{lensMap} &:: \text{BiGUL } a \ b \rightarrow \text{BiGUL } ([a], [b]) \ [b] \\ \text{lensMap } bx &= \text{lensFoldr } bx' (\lambda v \rightarrow \text{length } v \equiv 0) \\ &\quad \text{where } bx' = \$ (\text{rearrV } [\lambda (v : vs) \rightarrow (v, vs)]) \$ \\ &\quad \quad bx \text{ 'Prod' Replace} \end{aligned}$$

For testing, we define

$$\begin{aligned} \text{dec1} &:: \text{BiGUL } \mathbb{Z} \ \mathbb{Z} \\ \text{dec1} &= \text{emb } g \ p \\ &\quad \text{where } g \ s = s + 1 \\ &\quad \quad p \ s \ v = v - 1 \end{aligned}$$


```

*PList> put (lensMap dec1) ([0..10],[]) [100..110]
Right ([99,100,101,102,103,104,105,106,107,108,109],[])
*PList> get (lensMap dec1) ([1..10],[])
Right [2,3,4,5,6,7,8,9,10,11]
*PList> put (lensMap dec1) ([0..10],[]) [100]
Right ([99],[])
*PList> put (lensMap dec1) ([0..10],[]) [100..111]
Right ([99,100,101,102,103,104,105,106,107,108,109],[111])

```

4 Bidirectional Programming on Sets

5 Into BiGUL’s Bidirectionality

We have been writing *put* programs, usually having a corresponding *get* in mind but not explicitly describing it, and yet BiGUL is capable of finding the right *get* behaviour as if reading our mind. How? We will see that, when writing a BiGUL program, we are always simultaneously describing both a *put* function and a *get* function, which are guaranteed to be a well-behaved pair. And the “mind-reading” ability is far from magic: Well-behavedness directly implies that *get* is uniquely determined by *put*, which is the main motivation for designing a putback-based language. We will look at several constructs of BiGUL in detail to get a taste of such design.

5.1 Lenses, well-behavedness, and the fundamental theorem

Formally, we call a well-behaved pair of *put* and *get* a *lens*:

Definition 1 (lens). A lens between a source type s and a view type v consists of two functions:

$$\begin{aligned} \text{put} &:: s \rightarrow v \rightarrow \text{Maybe } s \\ \text{get} &:: s \rightarrow \text{Maybe } v \end{aligned}$$

satisfying two well-behavedness laws:

$$\begin{aligned} \text{put } s \ v = \text{Just } s' &\Rightarrow \text{get } s' = \text{Just } v && (\text{PUTGET}) \\ \text{get } s = \text{Just } v &\Rightarrow \text{put } s \ v = \text{Just } s && (\text{GETPUT}) \end{aligned}$$

In the original formulation [1], a lens refers to just a pair of functions having the right types, and one needs to explicitly say “well-behaved lens” to mean a well-behaved pair; we will talk about well-behaved lenses only, though, so we build well-behavedness into our definition of lenses. Also note that this definition models partial transformations explicitly as *Maybe*-valued functions: *put* and *get* are *total* functions that can nevertheless produce *Nothing* to indicate failure.

From this definition of well-behavedness, we can immediately prove what might be called the “fundamental theorem” of putback-based bidirectional programming:

Theorem 1 (uniqueness of *get*). *Given two lenses whose *put* components are equal, their *get* components are also equal.*

Proof. Let l and r be two lenses; denote their *put/get* components as *put l/get l* and *put r/get r* respectively, and assume that *put l* = *put r*. Then for any s and v ,

$$\begin{aligned}
& \textit{get } l \ s = \textit{Just } v \\
& \Leftrightarrow \{ \text{well-behavedness of } l \} \\
& \textit{put } l \ s \ v = \textit{Just } s \\
& \Leftrightarrow \{ \textit{put } l = \textit{put } r \} \\
& \textit{put } r \ s \ v = \textit{Just } s \\
& \Leftrightarrow \{ \text{well-behavedness of } r \} \\
& \textit{get } r \ s = \textit{Just } v
\end{aligned}$$

(This also entails that *get l s* = *Nothing* if and only if *get r s* = *Nothing*.) $\square$

The theorem guarantees that the BiGUL programmer is in full control of the bidirectional behavior — programming the *put* behavior is sufficient to determine the *get* behavior. Also, to the language designer, the theorem gives a kind of reassurance that, once the *put* behavior of a construct is determined, there is no need to worry about which *get* behavior should be adopted — there is at most one possibility. This is in contrast to *get*-based design, in which there are usually more than one viable *put* semantics that can be assigned to a *get*-based construct, and the designer needs to justify the choice or provide several versions.

5.2 Designing putback-based language components

Each BiGUL construct is conceived, at the design stage, as a lens (like *Skip* and *Replace*) or a lens *combinator* (like *Case*), which constructs a more complex lens from simpler ones. The *put* and *get* components of these lenses usually have to be developed together, but for each lens we will employ a more “*put*-oriented” design process: We start from an intended *put* behavior, and then add restrictions so that we can find a corresponding *get*. This does not guarantee that the lenses we arrive at will have a “strong *put* flavor” — that is, some of the lenses will be as suitable for *get*-based programming as for putback-based programming (or even more suitable). But we will also see that some other lenses are more naturally understood in terms of their *put* behavior.

Replace. The simplest lens is probably *Replace*, which replaces the entire source with the view:

$$\textit{put } \textit{Replace} \ s \ v = \textit{Just } v$$

Is there a *get* semantics that can be paired with this *put*? Yes, quite obviously — in fact, PUTGET directly gives us the definition of *get Replace*:

get Replace $v = \text{Just } v$

We still need to verify GETPUT, which can be easily checked to be true.

Skip. Coming up next is *Skip*, whose natural behavior is

put Skip $s \ v = \text{Just } s$

Considering PUTGET, though, we immediately see that this behavior is too liberal: If the view is simply thrown away, how can *get Skip* possibly recover it? One way out is to require that the view is trivial enough such that it can be thrown away and still be recovered, by setting the view type of *Skip* to the unit type $()$. Then it is easy for *get Skip* to recover the view, for which there is only one choice:

get Skip $s = \text{Just } ()$

This is the approach adopted prior to BiGUL 1.0.

More generally, we can establish well-behavedness as long as *get Skip* has only one view choice for each source, regardless of what the view type is. The existence of this “unique choice” is witnessed by a function $f :: s \rightarrow v$, which we add as an additional argument to *Skip*. The *get* direction is then

get (Skip f) $s = \text{Just } (f \ s)$

From the *put* direction, we may think of this function f as specifying a consistency relation, saying that the view information is completely included in the source (since you can compute the view from the source) and can be safely discarded. *Skip f* can be used if and only if the source and view are consistent in that sense. We thus arrive at:

put (Skip f) $s \ v = \text{if } v \equiv f \ s \ \text{then return } s \ \text{else Nothing}$

This pair of *put* and *get* can be verified to be well-behaved. *Skip f*, which features in BiGUL 1.0, is one lens which turns out to be more easily understood from the *get* direction — it bidirectionalizes any function whose codomain has decidable equality, albeit trivially. We get the first version of *Skip* as a special case by setting f to *const* $()$.

Prod. For a simplest example of a lens combinator, we look at *Prod*. Both the source and view types should be pairs; *Prod* accepts two lenses, say l and r , and applies them respectively to the left and right components:

put (l `Prod` r) $(sl, sr) \ (vl, vr) = \text{do } sl' \leftarrow \text{put } l \ sl \ vl$
 $sr' \leftarrow \text{put } r \ sr \ vr$
 $\text{return } (sl', sr')$

The *get* direction is unsurprising:

$$\begin{aligned} \text{get } (l \text{ 'Prod' } r) (sl, sr) &= \mathbf{do} \text{ } vl \leftarrow \text{get } l \text{ } sl \\ &\quad vr \leftarrow \text{get } r \text{ } sr \\ &\quad \text{return } (vl, vr) \end{aligned}$$

Having constructed *put* and *get* from *l* and *r*, we also expect that their well-behavedness is a consequence of the well-behavedness of *l* and *r*. Intuitively, this pair of *put* and *get* does look well-behaved if *l* and *r* are. But how do we establish that formally? To prove PUTGET, for example, we should prove from the assumption

$$\text{put } (l \text{ 'Prod' } r) (sl, sr) (vl, vr) = \text{Just } (sl', sr') \quad (1)$$

the conclusion

$$\text{get } (l \text{ 'Prod' } r) (sl', sr') = \text{Just } (vl, vr) \quad (2)$$

Both equations say that a somewhat complicated monadic *Maybe*-program computes successfully to some value. It may seem that we need some messy case analysis, but what we know about *Maybe*-programs tells us that such a program computes successfully if and only if every step of the program does, and this helps us to break both (1) and (2) into simpler equations. Formally, we have this lemma:

Lemma 1. *Let $mx :: \text{Maybe } a$ and $f :: a \rightarrow \text{Maybe } b$. Then, for all $y :: b$,*

$$mx \gg= f = \text{Just } y$$

if and only if

$$mx = \text{Just } x \quad \text{and} \quad f \text{ } x = \text{Just } y \quad \text{for some } x :: a$$

Proof. Case analysis on *mx*. □

This lemma can be nicely applied to *Maybe*-programs written in the **do**-notation, transforming such programs into *predicates* saying that a program computes to some given value. To do it more formally: Define a translation $\mathcal{S}$ from **do**-blocks of type *Maybe a* to predicates on *a* by

$$\begin{aligned} \mathcal{S} (\mathbf{do} \{x \leftarrow mx; B\}) y &= \exists x. mx = \text{Just } x \wedge \mathcal{S} (\mathbf{do} B) y \\ \mathcal{S} (\mathbf{do} \{my\}) y &= my = \text{Just } y \end{aligned}$$

Then we can extend Lemma 1 to the following:

Lemma 2. *The proposition*

$$\mathcal{S} (\mathbf{do} B) y$$

is true if and only if

$$\mathbf{do} B = \text{Just } y$$

Proof. By induction on the list structure of *B*, using Lemma 1 repeatedly. □

For example, applying $\mathcal{S}$ to $put\ (l\ 'Prod'\ r)\ (sl, sr)\ (vl, vr)$ yields

$$\begin{aligned} \lambda(sl'', sr'').\ \exists sl'.\ put\ l\ sl\ vl &= Just\ sl' \wedge \\ \exists sr'.\ put\ r\ sr\ vr &= Just\ sr' \wedge \\ return\ (sl', sr') &= Just\ (sl'', sr'') \end{aligned}$$

where the last equation is equivalent to $sl' = sl'' \wedge sr' = sr''$ (since $return = Just$ for the *Maybe* monad). Applying Lemma 2 and doing some simplification, (1) is equivalent to

$$put\ l\ sl\ vl = Just\ sl' \quad \wedge \quad put\ r\ sr\ vr = Just\ sr'$$

Similarly, (2) can be shown to be equivalent to

$$get\ l\ sl' = Just\ vl \quad \wedge \quad get\ r\ sr' = Just\ vr$$

The entailment is then just PUTGET for l and r .

Case. This is a representative combinator in BiGUL, and arguably the most complex one. For simplicity, let us consider a two-branch variant of *Case* first. A branch is a condition and a body; since in *put* we manipulate both a source and a view, the conditions in general can be binary predicates on both the source and view. We thus define the type of branches as:

$$\mathbf{type}\ CaseBranch\ s\ v = (s \rightarrow v \rightarrow Bool, BiGUL\ s\ v)$$

and consider the following variant of *Case*:

$$Case :: CaseBranch\ s\ v \rightarrow CaseBranch\ s\ v \rightarrow BiGUL\ s\ v$$

The straightforward behavior is

$$\begin{aligned} put\ (Case\ (pl, l)\ (pr, r))\ s\ v &= \mathbf{if}\quad pl\ s\ v\ \mathbf{then}\ put\ l\ s\ v \\ &\quad \mathbf{else\ if}\ pr\ s\ v\ \mathbf{then}\ put\ r\ s\ v \\ &\quad \mathbf{else}\ Nothing \end{aligned}$$

That is, depending on which condition is satisfied (with pl having higher priority), we execute either $put\ l$ or $put\ r$, or fail the computation if neither of the conditions is satisfied. Now, again, we ask the question: Can we find a *get* behavior to pair with this *put*?

Ruling out branch switching for PUTGET. An important working assumption here is that we want lens combinators to be *compositional*: When we looked at *Prod*, for example, we defined its *put* and *get* in terms of those of the smaller lenses, and derived the overall well-behavedness from that of the smaller lenses. For *Case*, this implies that, when establishing well-behavedness, we want a *get* following a *put* (or a *put* following a *get*) to use the same branch taken by the

put (or the *get*), so we can invoke PUTGET (or GETPUT) of the branch. The current *put* behavior of *Case* does not leave any clue in the updated source about which branch is used to produce it, though, so it is impossible for *get* to choose the right branch.

One solution, which does not require changing the syntax of *Case*, is to check that the ranges of the branches are *disjoint*. In general, for a lens, the range of a *put* can be shown to coincide with the domain of the corresponding *get*. So the *get* behavior of *Case* can simply try to execute both branches on the input source, and there will be at most one branch that computes successfully. We can put (expensive) disjointness checks into *put* such that if *put* succeeds, the subsequent *get* will have at most one branch to choose:

```

put (Case (pl, l) (pr, r)) s v =
  if    pl s v then do s' ← put l s v
                                maybe (return s') (const Nothing) (get r s')
  else if pr s v then do s' ← put r s v
                                maybe (return s') (const Nothing) (get l s')
  else Nothing

```

If *get* favors the first branch, meaning that it declares success as soon as the first branch succeeds (without requiring that the second branch fails),

```

get (Case (pl, l) (pr, r)) s = maybe (get r s) return (get l s)

```

then we can also omit the check in *put*'s first branch:

```

put (Case (pl, l) (pr, r)) s v =
  if    pl s v then put l s v
  else if pr s v then do s' ← put r s v
                                maybe (return s') (const Nothing) (get l s')
  else Nothing

```

Ruling out branch switching for GETPUT. The GETPUT direction, on the other hand, still does not avoid branch switching — the outcome of *get* does not say anything about which of *pl* and *pr* will be satisfied in the subsequent *put*. So we add some checks to *get* such that *get*'s success will tell us which branch will be chosen by *put*:

```

get (Case (pl, l) (pr, r)) s = maybe (do v ← getBranch (pr, r) s
                                     if pl s v then Nothing
                                     else return v)
                                     return
                                     (getBranch (pl, l) s)
getBranch (pl, l) s = do v ← get l s
                      if pl s v then return v
                      else Nothing

```

The definition of *put* should also be revised to use *getBranch* for disjointness check. But this breaks PUTGET! Since *put* does not guarantee that the updated source and the view satisfy the condition of the branch executed, even though *get* will be able to choose the right branch, the subsequent, newly added check is not guaranteed to succeed. We thus also need to add similar checks to *put*:

```

put (Case (pl, l) (pr, r)) s v =
  if    pl s v then do s' ← put l s v
                                if pl s' v then return s'
                                else Nothing
  else if pr s v then do s' ← put r s v
                                if pr s' v then maybe (return s')
                                                         (const Nothing)
                                                         (getBranch (pl, l) s')
                                else Nothing
  else Nothing

```

Now this pair of *put* and *get* can be verified to be well-behaved.

Improving the efficiency of get. The efficiency of the current *get* does not seem very good, especially when, in general, any number of branches is allowed, and *get* has to try to execute each branch, possibly with a high cost, until it reaches a successful one; also, inefficient *get* affects the efficiency of *put*, which calls *get* to check range disjointness. An idea is to ask the programmer to make a rough “prediction” of the range of each branch: We enrich *CaseBranch* with a third component, which is a source predicate:

```

type CaseBranch s v = (s → v → Bool, BiGUL s v, s → Bool)

```

This new predicate is supposed to be satisfied by the updated source; we again add checks to *put* to ensure this:

```

put (Case (pl, l, ql) (pr, r, qr)) s v =
  if    pl s v then do s' ← put l s v
                                if pl s' v ∧ ql s' then return s'
                                else Nothing
  else if pr s v then do s' ← put r s v
                                if pr s' v ∧ qr s'
                                then maybe (return s')
                                                         (const Nothing)
                                                         (getBranch (pl, l, ql) s')
                                else Nothing
  else Nothing

```

Let us call *pl* and *pr* the *main conditions*, and *ql* and *qr* the *exit conditions*. The exit condition, in general, over-approximates the range of a branch. In other words, in the *get* direction, every source in the domain of a branch satisfies the


```

else if  $pr\ s\ v$  then do  $s' \leftarrow put\ r\ s\ v$ 
    if  $pr\ s'\ v \wedge qr\ s'$ 
        then maybe (return  $s'$ )
            (const Nothing)
            (getBranch ( $pl, l, ql$ )  $s'$ )
        else Nothing
    else if  $pa\ s\ v$  then cont ( $f\ s\ v$ )
    else Nothing

```

What about *get*? It turns out that *get* can simply ignore the adaptive branch! If you have doubt about this “choice”, just invoke the fundamental theorem (Theorem 1): The *put* behavior is exactly what we want, and we can verify that the pair of *put* and *get* is well-behaved, so we are reassured that our “choice” is “correct”, simply because there is no other choice of *get*.

To sum up, we have arrived at a simpler variant of *Case* which nevertheless has all the features of the multi-branch *Case* in BiGUL. We have inserted various dynamic checks into the *put* semantics, and the BiGUL programmer needs to be aware of these constraints to make execution of *Case* succeed: For each normal branch, (i) the main condition should be satisfied after the update, (ii) the main conditions of the branches before this one should not be satisfied after the update, and (iii) the exit condition should be satisfied by the updated source. Also the ranges of all the normal branches should be disjoint; the programmer is encouraged to write disjoint exit conditions, which imply disjointness of the ranges, and improve the efficiency of *get*. Also, for each adaptive branch, the adapted source and the view should match the main condition of a normal branch.

RearrV. Source and view rearrangements are also among the more complex constructs of BiGUL. Their complexity lies in the strongly and generically typed treatment of pattern matching, though, rather than their bidirectional behavior. The two kinds of rearrangements are fairly similar, and we will discuss view rearrangement only.

Pattern matching is inherently a bidirectional operation: In one direction, we break something into a collection of its components at the variable positions of a pattern. This collection can be considered as indexed by the variable positions, and acting like an *environment* for expression evaluation. Indeed, conversely, if we have a pattern and a corresponding environment, we can treat the pattern as an expression and evaluate it under the environment. These two directions are inverse to each other, i.e., they form an isomorphism. For the language designer, it may be slightly tedious to establish such isomorphisms, but for the programmer, pattern matching and evaluation are arguably the most natural way to decompose and rearrange things. Previous bidirectional languages usually provide theoretically simpler combinators for decomposition and rearrangement, but they are hard to use in practice. BiGUL’s support of pattern matching, on the other hand, turns out to be one important contributing factor in its usability.

BiGUL's patterns are strongly typed: The programmer has to declare a target type for a pattern, and the pattern is guaranteed by typechecking to make sense for that target type. This can be achieved by defining the datatype of patterns as a generalised algebraic datatype:

```
data Pat a where
  PVar   :: Eq a => Pat a
  PConst :: Eq a => a -> Pat a
  PProd  :: Pat a -> Pat b -> Pat (a, b)
  PLeft  :: Pat a -> Pat (Either a b)
  PRight :: Pat b -> Pat (Either a b)
  PIn    :: InOut a => Pat (F a) -> Pat a
```

A pattern can be a (nameless) variable, a constant, a product, a *Left* or *Right* injection (for the *Either* type), or a generic constructor, and its target type is given as the index in its type. So, for example, *PProd* cannot be used to match an *Either* value. The *InOut* typeclass contains the types which can be broken into a sum-of-products representation. For example, $[a]$ is an instance of *InOut*, and $F [a]$, an isomorphic sum-of-products representation of $[a]$, is *Either* $() (a, [a])$. The isomorphism is witnessed by

$$inn :: InOut a \Rightarrow F a \rightarrow a \quad \text{and} \quad out :: InOut a \Rightarrow a \rightarrow F a$$

These two functions will be used to define pattern matching and evaluation.

How do we define pattern matching? The result of a pattern matching, as we mentioned above, is an environment indexed by the variable positions of the pattern. Here we want a safe (but not necessarily efficient) representation of the environment type, in the sense that the indexes into the environment should be exactly the variable positions of the pattern, which is enforced statically by typechecking. In other words, this type depends on the pattern, and a way to compute this type is to encode it as a second index of the *Pat* datatype:

```
data Pat a env where
  PVar   :: Eq a => Pat a (Var a)
  PConst :: Eq a => a -> Pat a ()
  PProd  :: Pat a a' -> Pat b b' b'' -> Pat (a, b) (a', b')
  PLeft  :: Pat a a' -> Pat (Either a b) a'
  PRight :: Pat b b' -> Pat (Either a b) b'
  PIn    :: InOut a => Pat (F a) b c -> Pat a b
```

Notice that an environment type is just a product of *Var* types. We will talk about *Var* later, which is simply defined by

```
newtype Var a = Var a
```

Now we can define the pattern matching operation:

```
deconstruct :: Pat a env -> a -> Maybe env
deconstruct PVar      x      = return (Var x)
```

```

deconstruct (PConst c) x      = if c ≡ x then return () else Nothing
deconstruct (l'PProd' r) (x, y) = liftM2 (,) (deconstruct l x)
                                (deconstruct r y)

deconstruct (PLeft p) (Left x) = deconstruct p x
deconstruct (PLeft _) _       = Nothing
deconstruct (PRight p) (Right x) = deconstruct p x
deconstruct (PRight _) _       = Nothing
deconstruct (PIn p) x          = deconstruct p (out x)

```

and its inverse (which is total):

```

construct :: Pat a env → env → a
construct PVar (Var x) = x
construct (PConst c) _ = c
construct (l'PProd' r) (envl, envr) = (construct l envl, construct r envr)
construct (PLeft p) env = Left (construct p env)
construct (PRight p) env = Right (construct p env)
construct (PIn p) env = inn (construct p env)

```

Now consider view rearrangement, which evaluates a “simple” pattern-matching λ -expression on the view and continue updating with the transformed view. The body of the λ -expression refers to the variables appearing in the pattern. How do we represent such references? We have seen that an environment type is a product, i.e., a binary tree; to refer to a component in an environment, we can use a *path* that goes from the root to a sub-tree. In BiGUL, these paths are called *directions*:

```

data Direction env a where
  DVar  :: Direction (Var a) a
  DLeft :: Direction a t → Direction (a, b) t
  DRight :: Direction b t → Direction (a, b) t

```

The type of a direction is indexed by the environment type it points into and the component type it points to. Note that the type of *DVar* is specified to work with only environment types marked with *Var*; this is for ensuring that a direction goes all the way down to an actual component at a variable position of the pattern, rather than stopping half-way, pointing to a sub-tree which include more than one component. It is easy to extract a component from an environment following a direction:

```

retrieve :: Direction env a → env → a
retrieve DVar (Var x) = x
retrieve (DLeft d) (x, _) = retrieve d x
retrieve (DRight d) (_, y) = retrieve d y

```

Now we can define *expressions*, which are similar to patterns but include directions rather than variables, to represent the body of rearranging λ -expressions:

```

data Expr env a where
  EDir  :: Direction env a → Expr env a
  EConst :: (Eq a) ⇒ a → Expr env a
  EProd  :: Expr env a → Expr env b → Expr env (a, b)
  ELeft  :: Expr env a → Expr env (Either a b)
  ERight :: Expr env b → Expr env (Either a b)
  EIn    :: (InOut a) ⇒ Expr env (F a) → Expr env a

```

Evaluating an expression under an environment is similar to inverse pattern matching:

```

eval :: Expr env a → env → a
eval (EDir d)    env = retrieve d env
eval (EConst c)  env = c
eval (l `EProd` r) env = (eval l env, eval r env)
eval (ELeft e)   env = Left  (eval e env)
eval (ERight e)  env = Right (eval e env)
eval (EIn e)     env = inn  (eval e env)

```

The type of *RearrV* is then:

```

RearrV :: Pat v env → Expr env v' → BiGUL s v' → BiGUL s v

```

And its *put* behavior is simply:

```

put (RearrV p e b) s v = do env ← deconstruct p v
                        put b s (eval e env)

```

For the *get* direction, after executing the inner BiGUL program to obtain an intermediate view, we should reverse the roles of the pattern and body in the λ -expression, using the latter as a pattern to match the intermediate view. This intermediate view will be decomposed, and eventually each of its components will be paired with a direction indicating which variable position the component should go into. We can prepare a “container” shaped like the pattern, whose variable positions are initially empty. Whenever we reach a component and a direction, we try to put that component into the place in the container pointed to by the direction; if two components are put into the same position twice (indicating that the λ -expression uses a variable more than once), then they must be equal. In the end, all places in the container must be filled before it can be used as an environment to evaluate the pattern. Again, to compute the type of containers from a pattern, we add a third index to *Pat*:

```

data Pat a env con where
  PVar  :: Eq a ⇒ Pat a (Var a) (Maybe a)
  PConst :: Eq a ⇒ a → Pat a () ()
  PProd  :: Pat a a' a'' → Pat b b' b'' → Pat (a, b) (a', b') (a'', b'')
  PLeft  :: Pat a a' a'' → Pat (Either a b) a' a''

```

```

PRight :: Pat b b' b'' → Pat (Either a b) b' b''
PIn     :: InOut a ⇒ Pat (F a) b c → Pat a b c

```

A container type is just like an environment type except that the variable positions give rise to *Maybe* instead of *Var*. The first step — matching a value with an *expression* — can then be implemented as:

```

uneval :: Pat a env con → Expr env b → b → con → Maybe con
uneval p (EDir d)    x      con = unevalDir p d x con
uneval p (EConst c) x      con = if c ≡ x then return con else Nothing
uneval p (EProd l r) (x, y) con = uneval p l x con >>= uneval p r y
uneval p (ELeft e)   (Left x) con = uneval p e x con
uneval p (ELeft _)   x      con = Nothing
uneval p (ERight e)  (Right x) con = uneval p e x con
uneval p (ERight _)  x      con = Nothing
uneval p (EIn e)     x      con = uneval p e (out x) con

unevalDir :: Pat a env con → Direction env b → b → con → Maybe con
unevalDir PVar      DVar      x (Just y) = if x ≡ y
                                           then return (Just x)
                                           else Nothing
unevalDir PVar      DVar      x Nothing  = return (Just x)
unevalDir (PConst c) _        x con      = return con
unevalDir (l' PProd' r) (DLeft d) x (conl, conr) = liftM (, conr)
                                           (unevalDir l d x conl)
unevalDir (l' PProd' r) (DRight d) x (conl, conr) = liftM (conl, )
                                           (unevalDir r d x conr)
unevalDir (PLeft p)   d      x con      = unevalDir p d x con
unevalDir (PRight p)  d      x con      = unevalDir p d x con
unevalDir (PIn p)     d      x con      = unevalDir p d x con

```

This function *uneval* initially takes an empty container, which is generated by:

```

emptyContainer :: Pat v env con → con
emptyContainer PVar      = Nothing
emptyContainer (PConst c) = ()
emptyContainer (l' PProd' r) = (emptyContainer l, emptyContainer r)
emptyContainer (PLeft p)   = emptyContainer p
emptyContainer (PRight p)  = emptyContainer p
emptyContainer (PIn p)     = emptyContainer p

```

And then we can try to convert a container to an environment, checking whether the container is full in the process:

```

fromContainerV :: Pat v env con → con → Maybe env
fromContainerV PVar      Nothing  = Nothing
fromContainerV PVar      (Just v) = return (Var v)
fromContainerV (PConst c) con    = return ()

```

$$\begin{aligned}
& \text{fromContainerV } (l \text{ 'PProd' } r) \text{ (conl, conr)} = \text{liftM2 } (,) \\
& \hspace{15em} (\text{fromContainerV } l \text{ conl }) \\
& \hspace{15em} (\text{fromContainerV } r \text{ conr }) \\
& \text{fromContainerV } (P\text{Left } p) \text{ con} = \text{fromContainerV } pat \text{ con} \\
& \text{fromContainerV } (P\text{Right } p) \text{ con} = \text{fromContainerV } pat \text{ con} \\
& \text{fromContainerV } (P\text{In } p) \text{ on} = \text{fromContainerV } pat \text{ con}
\end{aligned}$$

If we can get hold of an environment, then we can let out a sigh of relief, since the last step — inverse pattern matching — is total. To sum up:

```

get (RearrV p e b) s = do v' ← get b s
                      con ← uneval p e v' (emptyContainer p)
                      env ← fromContainerV p con
                      return (construct p env)

```

Conceptually, this is just reversing pattern matching and expression evaluation. To actually prove the well-behavedness, though, we need to reason about stateful computation (which is what *uneval* essentially is), which involves coming up with suitable invariants and proving that they are maintained throughout the computation. It is somewhat tedious, but can be done without a problem.

It is interesting to mention that there would be a catch if we designed this combinator from the *get* direction: It is tempting to think that a rearranging λ -expression gives rise to an isomorphism, which can be lifted to a lens, and can be composed with the inner lens to give a lens semantics to *RearrV*. This would result in a redundant computation of an intermediate source which is immediately discarded, and now the success of the whole computation would unnecessarily depend on that of the intermediate source. We would need to use a special composition which composes a lens with an isomorphism on the right, but such a need would be hard to notice since the *get* behavior of the two compositions are the same — we really have to think in terms of *put* to see that the special composition is needed.

References

1. Foster, J.N., Greenwald, M.B., Moore, J.T., Pierce, B.C., Schmitt, A.: Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem. *ACM Transactions on Programming Languages and Systems* 29(3), 17 (2007)